

Session 8: Sequence Models & Time-Series Forecasting

Javier Serrano

Applied Machine Learning
Master in Data Science & Analytics

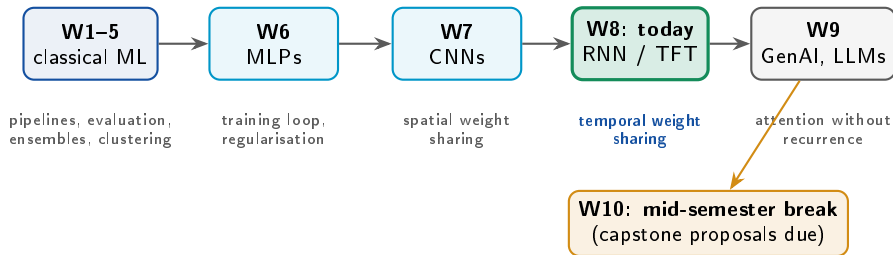


Strathmore University

@iLabAfrica Centre

Where We Are in the Course

Part II: deep learning, week three



Today we give our networks a notion of *time*: the data are ordered, and the order itself carries signal.

This session is the basis of **Chapter 8** of the course book.

Learning Objectives

By the end of this session you will be able to ...

1. **Frame** a forecasting problem as supervised learning: windows, horizons, and the three covariate families (static, past-observed, known-future).
2. **Evaluate** temporal models correctly: rolling-origin backtesting, scale-free metrics (MASE, sMAPE) and the quantile (pinball) loss.
3. **Derive and implement** LSTM and GRU cells in `tf.keras`, reading gates as learned read, write and forget operations on memory.
4. **Explain** the Temporal Fusion Transformer: gated residual networks, variable selection, and interpretable multi-head attention.
5. **Design** a production forecasting system for agent liquidity in a Kenyan mobile-money network, with its governance obligations.

Roadmap

Temporal Data in Applied ML

Forecasting as Supervised Learning

Recurrent Neural Networks: LSTM and GRU Cells

Temporal Fusion Transformers

Case Study: Agent Liquidity Forecasting

Recap and Readings

Everything the Platform Ingests Is a Stream

Four temporal signals in our running case study

Transaction volumes

payroll spikes,
market days

Agent float

cash vs. e-value,
rebalanced daily

Repayment behaviour

trajectories,
not averages

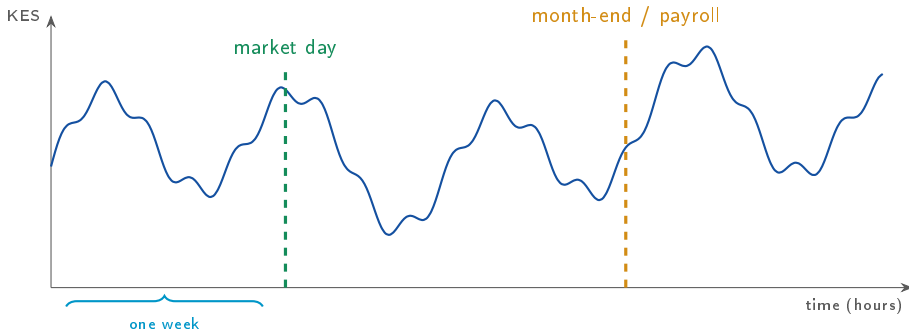
Fraud

bursts, velocity,
device churn

A static tabular snapshot throws the ordering away.
Sequence models consume it.

What an Hourly Agent Series Looks Like

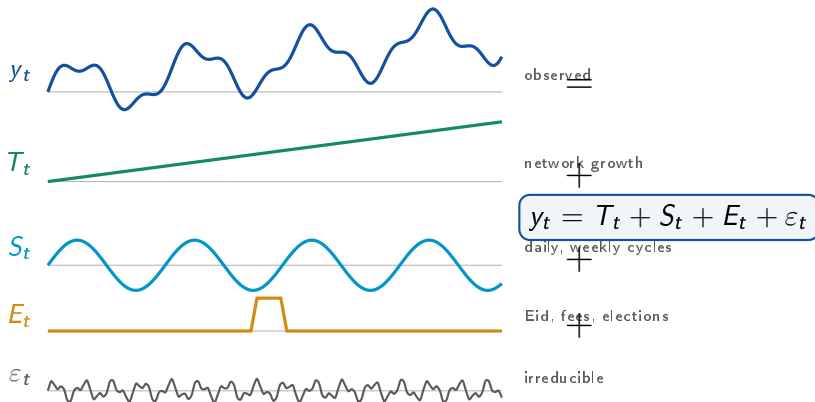
Daily cycle, weekly cycle, calendar events



Three structures are stacked here: a slow **trend**, nested **cycles** (hour of day, day of week), and **calendar events**.

Anatomy of a Time Series

The classical additive decomposition



Deep models learn T , S and E jointly, but you should still *feed them the calendar* as explicit features.

The Forecasting Task, Formally

Notation for the rest of the course

Given history up to origin t , predict a **horizon** of H future values:

$$\hat{y}_{t+1:t+H} = f_{\theta} \left(\underbrace{y_{t-L+1:t}}_{\text{targets, lookback } L}, \underbrace{z_{t-L+1:t}}_{\text{past-observed}}, \underbrace{x_{t-L+1:t+H}}_{\text{known-future}}, \underbrace{s}_{\text{static}} \right)$$

- L is chosen from the autocorrelation structure, H from the *decision* being supported.
- The three covariate families are not pedantry: they differ in *when* the information becomes available, and a model that ignores the difference will leak.

Decision first. For agent rebalancing planned each evening, $H = 24$ hours. The horizon is an operational fact, not a hyperparameter.

Stationarity: The Vocabulary You Still Need

Classical assumptions, modern practice

- A series is (weakly) **stationary** if its mean, variance and autocovariance do not depend on t . Most business series are not stationary: trend and seasonality both violate the condition.
- Classical models (the ARIMA family) **difference** the series to enforce it:

$$\Delta y_t = y_t - y_{t-1}, \quad \Delta_m y_t = y_t - y_{t-m} \text{ (seasonal lag } m\text{)}.$$

- Deep models do not *require* stationarity, but they train far better on stabilised inputs. Our standard recipe for KES volumes with heavy right tails:

$$y_t \mapsto \log(1 + y_t) \mapsto \text{per-series standardisation on the training span only.}$$

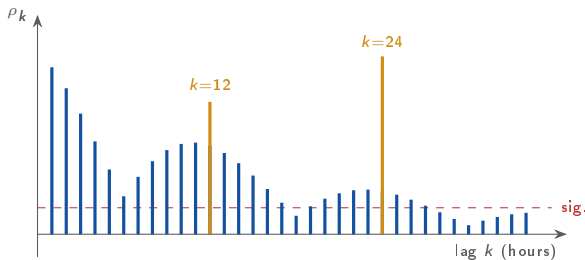
Autocorrelation: The Cheapest Feature Selection You Own

Read the lookback off the ACF plot

Autocorrelation at lag k :

$$\rho_k = \frac{\text{Cov}(y_t, y_{t-k})}{\text{Var}(y_t)}$$

- Spikes at $k = 24$ and $k = 168$ (hourly data) mean daily and weekly cycles.
- They tell you the lookback L and the seasonal lag m *before* you train anything.



Stylised ACF of hourly agent volume: decay plus periodic spikes.

Always Beat the Boring Baselines

If your LSTM cannot beat seasonal naïve, ship seasonal naïve

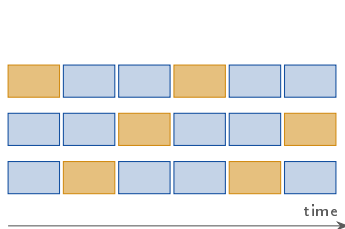
Baseline	Rule	When it is strong
Naïve	$\hat{y}_{t+h} = y_t$	random-walk-like series
Seasonal naïve	$\hat{y}_{t+h} = y_{t+h-m}$	the one to beat for agent volumes
ETS / EWMA	recursive weighted average	short series, smooth demand
(S)ARIMA	linear model on differences	low frequency, regulator-friendly
LightGBM on lags	tabular model, lag and calendar features	many series, rich covariates

Pitfall. Many published “deep forecasting beats X” results evaporate against a properly tuned seasonal-naïve or ETS baseline. In a fintech production review the first question is always: *by how much do you beat seasonal naïve, at which horizons?*

Backtesting, Not Random Cross-Validation

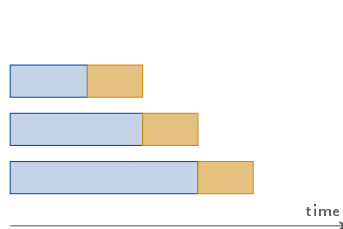
The single most common leakage bug in industry

Random k -fold: wrong



future blocks land in training:
every metric is inflated

Rolling origin: correct



the test block always lies
strictly after the training block

scikit-learn's `TimeSeriesSplit` implements the right-hand picture. Fit scalers and encoders *inside* each fold, and keep a final untouched out-of-time block for the last word.

Point Metrics That Survive Contact with Finance

Scale-free or nothing

On test pairs $(y_i, \hat{y}_i)$:

$$\text{MAE} = \frac{1}{n} \sum_i |y_i - \hat{y}_i|, \quad \text{sMAPE} = \frac{200}{n} \sum_i \frac{|y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|}, \quad \text{MASE} = \frac{\text{MAE}}{\text{MAE}_{\text{seasonal naïve}}}$$

- $\text{MASE} < 1$ means you beat seasonal naïve. Being scale-free, it is comparable across a Gikomba stall and a Westlands supermarket on the same dashboard.
- MAPE explodes near zero volumes (overnight hours), so prefer sMAPE or MASE.
- Always report metrics **per horizon step**: a model can be excellent at $h = 1$ and useless at $h = 24$, and it is $h = 24$ that pays for the rebalancing truck.

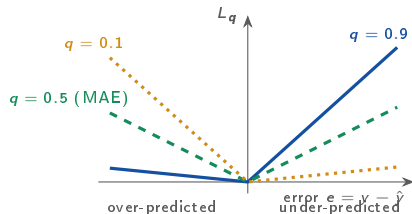
The Pinball Loss: Training for a Tail, Not a Mean

Where the business decision enters the loss function

For quantile $q \in (0, 1)$, with error
 $e = y - \hat{y}^{(q)}$:

$$L_q(y, \hat{y}^{(q)}) = \max(q e, (q - 1) e)$$

- Under-prediction is penalised with weight q , over-prediction with $1 - q$.
- Minimising L_q recovers the q -th conditional quantile.
- Training at $q = 0.9$ yields the level exceeded only one day in ten: exactly the safety-stock number a liquidity desk needs.



The asymmetry *is* the risk appetite: a steep right arm says stock-outs hurt more than idle float.

Choosing L , H and the Stride

Three decisions, three different sources of truth

Knob	Decided by	Agent-float example
Lookback L	the ACF: cover at least one full seasonal period, usually two	$L = 168$ hours (one week)
Horizon H	the operational decision the forecast feeds	$H = 24$ hours (tonight's rebalancing plan)
Stride	compute budget and redundancy	1 hour: windows overlap heavily, which is fine

- Longer L is not free: memory grows linearly for RNNs and quadratically for attention.
- If the ACF shows a spike at lag 168, an L of 48 cannot see it, whatever the architecture.

Windowing in tf.keras

One utility does the heavy lifting

```
import numpy as np, tensorflow as tf

L, H = 168, 24                                # lookback 1 week, horizon 1 day (hourly)
series = np.log1p(agent_df["volume_kes"].to_numpy("float32")) # tame the tail
# temporal split BEFORE windowing: never window across the boundary
n = len(series)
train, valid, test = series[: int(.7*n)], series[int(.7*n): int(.85*n)], series[int(.85*n):]

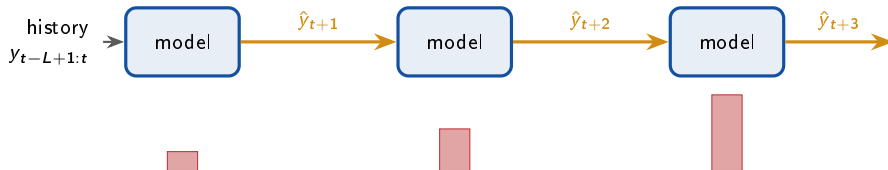
mu, sd = train.mean(), train.std() # scaler fitted on train only (leakage!)
scale = lambda a: (a - mu) / sd

def make_ds(arr, shuffle):
    return tf.keras.utils.timeseries_dataset_from_array(
        data=scale(arr)[: -H], targets=None, sequence_length=L + H,
        batch_size=256, shuffle=shuffle
    ).map(lambda w: (w[:, :L, None], w[:, L:])) # (X: [B,L,1], y: [B,H])

train_ds, valid_ds = make_ds(train, True), make_ds(valid, False)
```

Multi-Horizon Strategy 1: Recursive

One model, fed its own predictions



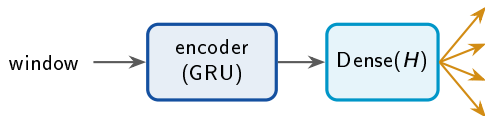
error compounds: each step is fed a prediction, not an observation

Cheap (one model for any horizon) and the classical choice, but the error bars widen faster than they should because the model never sees its own mistakes during training.

Multi-Horizon Strategies 2 and 3: Direct and Seq2Seq

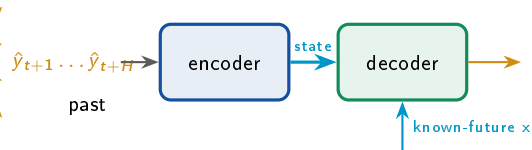
Predict the whole horizon in one forward pass

Direct



no error feedback,
simple, our default

Seq2seq



each future step gets its own covariates,
the skeleton inside the TFT

Practice note. For operational horizons of 24 to 72 hours, the *direct* strategy with a vector output is hard to beat for the effort involved. Reach for seq2seq or TFT when rich known-future covariates (promotions, holidays, payroll calendar) should shape each future step differently.

Roadmap

Temporal Data in Applied ML

Forecasting as Supervised Learning

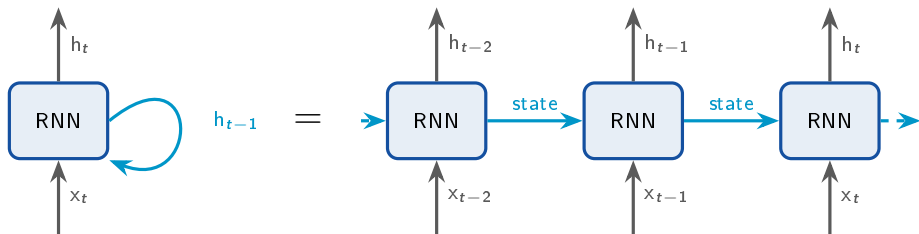
Recurrent Neural Networks: LSTM and GRU Cells

Temporal Fusion Transformers

Recap and Readings

The Recurrent Idea

A layer with a loop: state carried across time



the *same* weights W_x, W_h, b are reused at every step

$$h_t = \tanh(W_x^\top x_t + W_h^\top h_{t-1} + b)$$

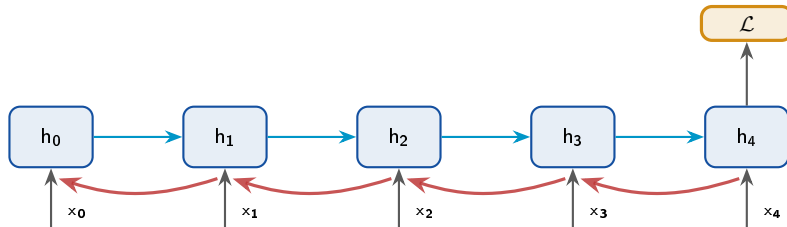
What the Hidden State Actually Is

Three consequences worth stating explicitly

- h_t is a **learned, lossy summary of everything seen so far**. Nothing constrains what it stores: the network decides, from the loss, which aspects of the past are worth carrying.
- The **parameter count is independent of the sequence length**. The same cell handles $L = 50$ or $L = 500$, which is exactly the temporal analogue of the spatial weight sharing you met in CNNs last week.
- Unrolled, an RNN is a very deep feed-forward network with *tied* weights. Every intuition you have about deep networks (initialisation, normalisation, gradient pathologies) transfers, and so do the problems.

Backpropagation Through Time

Ordinary backprop on the unrolled graph



gradients for the
shared weights are
summed over all
time steps

gradient flows backwards through every step

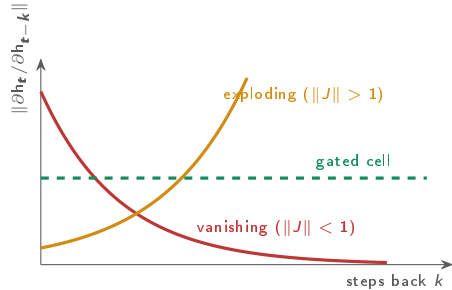
Unroll over L steps, run standard backprop on the resulting graph, then sum the gradients that reach each shared weight. Nothing new in principle: the trouble is in the arithmetic.

Why plain RNNs forget

The gradient at step t w.r.t. the state k steps earlier is a *product* of k Jacobians:

$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{j=t-k+1}^t \frac{\partial h_j}{\partial h_{j-1}}$$

- Norms below 1: the product **vanishes**, so distant steps receive no learning signal.
- Norms above 1: it **explodes** and training diverges.
- A plain RNN forgets beyond 10 to 50 steps, far short of our $L = 168$.



A gated cell keeps an additive path along which the gradient neither shrinks nor blows up.

Mitigations, in Increasing Order of Effectiveness

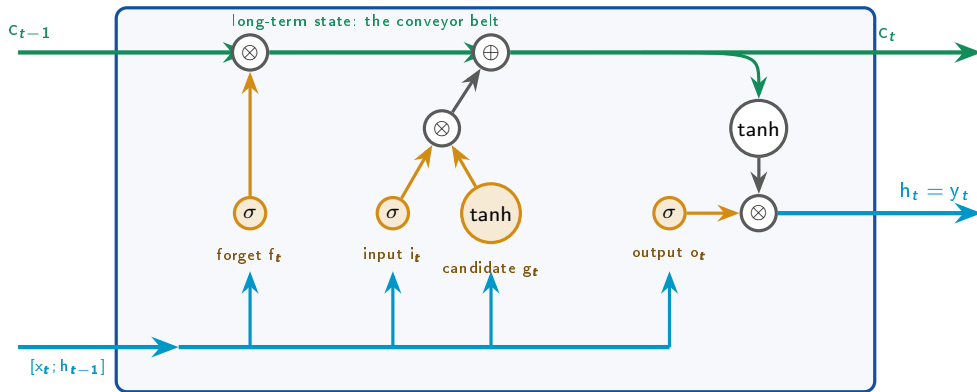
From plumbing fixes to a new cell

Fix	Addresses	Keras
Saturating activations	explosion	activation="tanh" (default)
Gradient clipping	explosion	Adam(clipnorm=1.0)
Layer normalisation	both	LayerNormalization
1D-conv front-end	shortens the effective sequence	Conv1D(padding="causal", strides=4)
Gated cells	vanishing, decisively	LSTM, GRU

Historical note. The vanishing-gradient analysis (Hochreiter 1991; Bengio et al. 1994) directly motivated the LSTM (Hochreiter and Schmidhuber, 1997). The fix arrived a decade and a half before deep learning became fashionable.

The LSTM Cell

A memory with learned read, write and forget operations



Two states flow through the cell: the **long-term state** c_t , touched only by element-wise gates, and the **short-term state** h_t .

LSTM Computations

Four small layers, three gates, one memory update

For each time step (Géron, Eq. 15–3):

$$i_t = \sigma(W_{xi}^\top x_t + W_{hi}^\top h_{t-1} + b_i)$$

input gate: what to write

$$f_t = \sigma(W_{xf}^\top x_t + W_{hf}^\top h_{t-1} + b_f)$$

forget gate: what to erase

$$o_t = \sigma(W_{xo}^\top x_t + W_{ho}^\top h_{t-1} + b_o)$$

output gate: what to reveal

$$g_t = \tanh(W_{xg}^\top x_t + W_{hg}^\top h_{t-1} + b_g)$$

candidate content

$$c_t = f_t \otimes c_{t-1} + i_t \otimes g_t$$

memory update (the additive path)

$$h_t = y_t = o_t \otimes \tanh(c_t)$$

exposed state

The update of c_t is *additive*, so $\partial c_t / \partial c_{t-1} = f_t$. With the forget gate near one, gradients travel back hundreds of steps almost unattenuated.

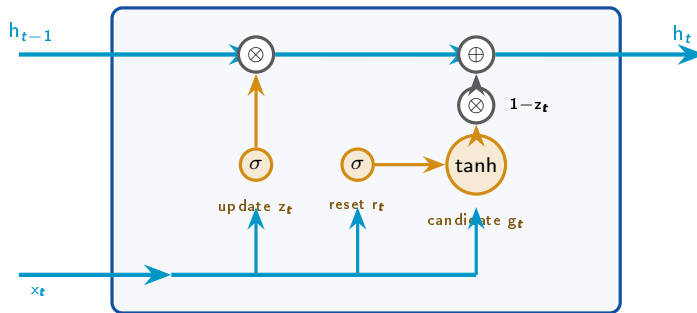
Reading the Gates as Memory Management

What the cell is doing, in fintech terms

- σ outputs lie in $(0, 1)$, so gates are *soft*, fully differentiable switches rather than hard branches. A value of 0.97 means “keep almost all of this”.
- Keras initialises $b_f = 1$, so an untrained cell starts by **remembering everything** and learns what to discard. Starting from forgetting would kill the gradient signal immediately.
- Concretely: the cell can hold “a month-end spike is approaching” in c_t across hundreds of hourly steps and release it precisely when the calendar features fire.
- A gate is also a *diagnostic*. Plotting mean forget-gate activation over a validation window shows where the model believes its memory should reset (typically at day boundaries).

The GRU Cell

The LSTM's leaner sibling: one state, two gates



One controller, two jobs.

z_t does the work of the LSTM's forget *and* input gates: whenever the cell writes, it erases first.

r_t decides how much of the old state the candidate may even see.

GRU Computations

Cho et al., 2014

$$z_t = \sigma(W_{xz}^\top x_t + W_{hz}^\top h_{t-1} + b_z)$$

update gate

$$r_t = \sigma(W_{xr}^\top x_t + W_{hr}^\top h_{t-1} + b_r)$$

reset gate

$$g_t = \tanh(W_{xg}^\top x_t + W_{hg}^\top (r_t \otimes h_{t-1}) + b_g)$$

candidate, gated view of the past

$$h_t = z_t \otimes h_{t-1} + (1 - z_t) \otimes g_t$$

convex mixture of old and new

- The final line is a **convex combination**: the GRU cannot write without erasing, which is precisely the coupling the LSTM leaves free.
- With $z_t \rightarrow 1$ the state is copied unchanged, giving the same protected additive path that saves the LSTM from vanishing gradients.

LSTM or GRU?

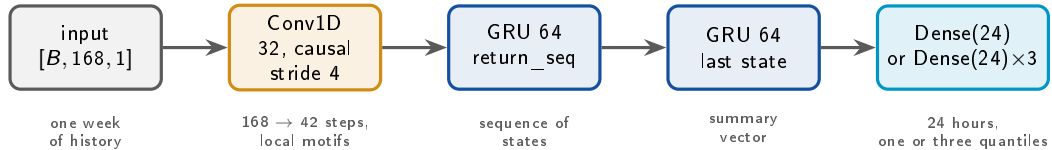
And what neither of them solves

	LSTM	GRU
States	2 (c_t, h_t)	1 (h_t)
Gates	3	2
Parameters per unit	$4(d_x + d_h + 1)d_h$	$3(d_x + d_h + 1)d_h$
Training speed	slower	roughly 25% faster
Accuracy in practice	roughly equivalent (Greff et al., 2015)	

- Course default: **start with the GRU**, and switch to an LSTM only if the validation backtest says so.
- Both still struggle beyond a few hundred steps: the state is a fixed-size bottleneck, and processing is strictly sequential, so there is no parallelism across time on a GPU.
- Two escape routes: a **1D convolutional front-end** to shorten the sequence, or **attention**, which is where the TFT enters.

The Architecture We Will Actually Train

Conv front-end, stacked GRU, direct horizon head



The convolution is the cheap win: it cuts the number of recurrent steps by a factor of four, so the GRU only has to remember across 42 positions instead of 168.

A Direct Multi-Horizon GRU Forecaster

The model above, in code

```
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(L, 1)),          # [batch, 168, 1]
    # 1D-conv front-end: shortens the sequence 4x, sees local motifs
    tf.keras.layers.Conv1D(32, kernel_size=6, strides=4,
                           padding="causal", activation="relu"),
    tf.keras.layers.GRU(64, return_sequences=True),
    tf.keras.layers.GRU(64),                      # last state only
    tf.keras.layers.Dense(H)                      # direct 24-step output
])

model.compile(
    loss=tf.keras.losses.Huber(),                 # robust to KES spikes
    optimizer=tf.keras.optimizers.Adam(1e-3, clipnorm=1.0),
    metrics=[tf.keras.metrics.MeanAbsoluteError(name="mae")]
)
```

Three deliberate choices. padding="causal" never peeks at the future; clipnorm tames exploding gradients; the Huber loss stops a single flash-crowd hour from dominating training.

Quantile Heads in tf.keras

One output and one loss per quantile

```
QS = [0.1, 0.5, 0.9]

def pinball(q):
    def loss(y, yhat):
        # y: [B,H], yhat: [B,H]
        e = y - yhat
        return tf.reduce_mean(tf.maximum(q * e, (q - 1.0) * e))
    return loss

inp = tf.keras.layers.Input(shape=(L, 1))
h = tf.keras.layers.GRU(64)(inp)
outs = [tf.keras.layers.Dense(H, name=f"q{int(q*100)}")(h) for q in QS]

qmodel = tf.keras.Model(inp, outs)
qmodel.compile(optimizer="adam", loss=[pinball(q) for q in QS])
```

The three heads share the recurrent trunk, so the extra cost is negligible. Nothing enforces

$\hat{y}^{(0.1)} \leq \hat{y}^{(0.5)} \leq \hat{y}^{(0.9)}$, so either add a monotonicity penalty or simply sort the outputs at inference time to prevent quantile crossing. This is exactly the output format of the TFT.

Design Caution: Bidirectional Layers and Forecasting

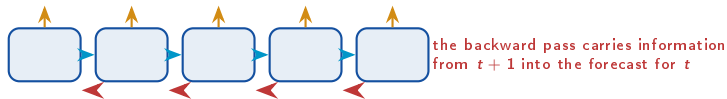
Legitimate for encoding, fatal for prediction

Fine: encoding a completed past window



every forecast
sees its own future:
leakage

Never: wrapping a layer that emits forecasts



`Bidirectional(LSTM(...))` is a reasonable encoder for a closed window, and a guaranteed leak anywhere downstream of the forecast origin.

Two More Traps: Scaling and Cold Starts

With 5,000 agents on one model

1. **Per-series scaling.** Standardise *each agent's* series using statistics from its own training span only, then give the model a static agent embedding. Otherwise the handful of largest agents dominates the loss and the model simply learns their shape.
2. **Cold starts and gaps.** New agents have no history and network downtime creates missing hours. Mask or impute explicitly, and add an `is_imputed` indicator channel: the model should know when it is looking at a guess rather than an observation.

Leakage checklist, temporal edition. Scaler fitted on the training span only · no window straddles a split boundary · causal padding everywhere · target-derived features lagged by at least the forecast latency · backtest folds strictly ordered · known-future covariates genuinely knowable in advance.

Roadmap

Temporal Data in Applied ML

Forecasting as Supervised Learning

Temporal Fusion Transformers

Recap and Readings

Why Go Beyond Recurrent Cells?

The requirements a production system adds

Heterogeneous inputs. Static metadata, past-observed series and known-future calendars need different treatment.

Multi-horizon quantiles in a single forward pass, not one model per horizon and per quantile.

Long-range dependencies. Last Ramadan's pattern is informative for this Ramadan, 8,000 steps back.

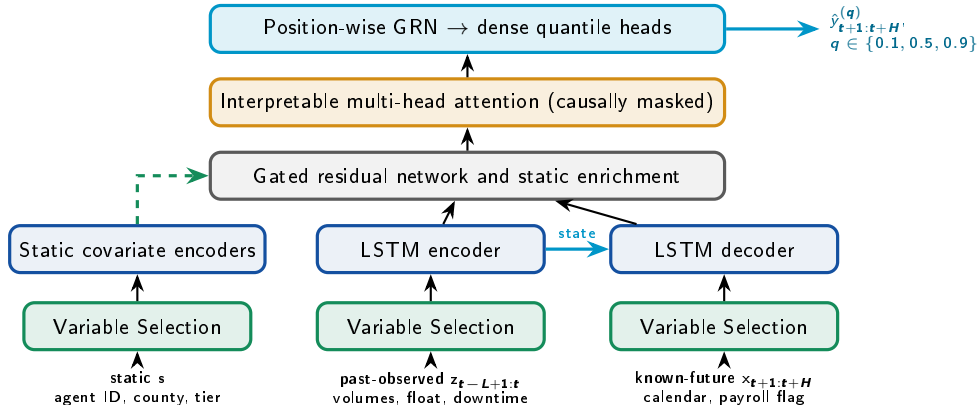
Interpretability. The model-risk reviewer will ask which inputs drove this forecast, and will expect an answer.

TFT in one sentence. A seq2seq model that uses an LSTM for *local* patterns, interpretable self-attention for *long-range* patterns, learned gating to *skip* anything unnecessary, and variable-selection networks to report *what mattered*, emitting quantile forecasts for every horizon step at once.

Lim, Arık, Loeff and Pfister, *International Journal of Forecasting* 37(4), 2021.

TFT Architecture

Follow one forward pass, bottom to top



Every component is skippable through its gate: on a simple dataset the TFT can learn to behave like a plain LSTM.

Reading the Architecture

Four design decisions, four blocks

Block	Problem it solves	What you get back
Variable selection networks	irrelevant and noisy inputs	instance-wise and global feature importance
Gated residual networks	unknown required depth	the model can collapse a block to the identity
LSTM encoder-decoder	local ordering and short cycles	positional structure without positional encodings
Interpretable attention	long-range dependencies	one readable attention map per forecast

The static stream is not a side input: it is injected into *every* other block as a context vector, so agent identity conditions selection, gating and attention alike.

Building Block 1: The Gated Residual Network

Apply nonlinearity only where the data demands it

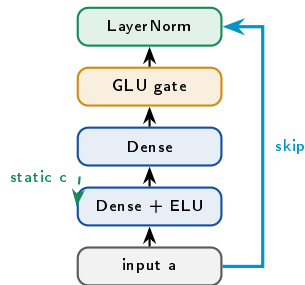
$$\eta_2 = \text{ELU}(W_2 a + W_3 c + b_2)$$

$$\eta_1 = W_1 \eta_2 + b_1$$

$$\text{GLU}(\gamma) = \sigma(W_4 \gamma + b_4) \otimes (W_5 \gamma + b_5)$$

$$\text{GRN}(a, c) = \text{LayerNorm}(a + \text{GLU}(\eta_1))$$

with a the primary input and c an optional static context vector.



- The **GLU gate** can drive its σ term to zero, collapsing the block to the identity. Unnecessary complexity is *learned away* rather than tuned away.
- Note the family resemblance to the LSTM and GRU gates: the same multiplicative switching, applied *depth-wise* instead of *time-wise*.

Building Block 2: Variable Selection Networks

Instance-wise feature importance, built into the architecture

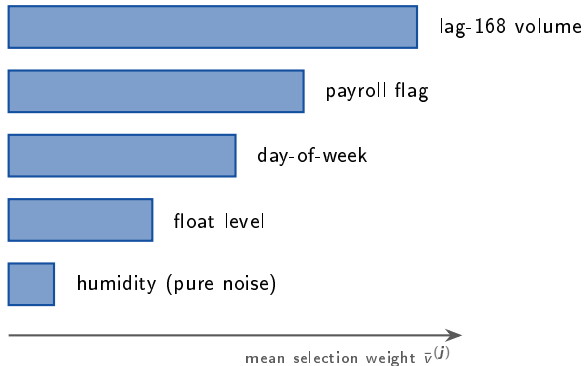
At each step every input variable j is embedded to $\xi_t^{(j)}$. A GRN then produces **selection weights** over the m variables, conditioned on the static context c_s :

$$\mathbf{v}_t = \text{softmax}(\text{GRN}(\Xi_t, c_s)) \in \mathbb{R}^m, \quad \tilde{\xi}_t = \sum_{j=1}^m v_t^{(j)} \text{GRN}_j(\xi_t^{(j)}).$$

- Noisy inputs receive weights near zero: this is **regularisation by architecture**, not by penalty.
- The weights are *instance-wise*, so a rural agent and a CBD agent can rely on different variables at the same hour.
- Averaging $\mathbf{v}_t$ over the test set yields a global variable-importance ranking at no extra cost.

What Variable Selection Reports Back

The chart that goes into the model documentation



How to read it.

The weekly lag dominates, the payroll calendar is second, and the injected noise variable is correctly ignored.

If a protected proxy climbed this chart, you would have a fairness finding, not a feature.

PLACEHOLDER: illustrative weights. Regenerate from the real backtest before distribution.

Building Block 3: Interpretable Multi-Head Attention

Long-range memory with a readable map

Standard scaled dot-product attention over the LSTM-processed sequence, with a **causal mask** so each forecast position attends only to the past:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_{\text{attn}}}}\right) \mathbf{V}$$

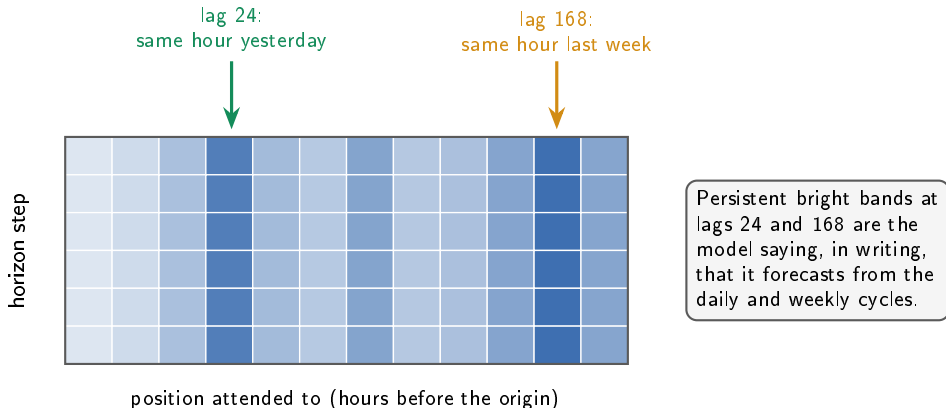
The TFT's modification: all heads **share the value projection** $\mathbf{W}_V$ and their outputs are averaged,

$$\tilde{\mathbf{A}} = \frac{1}{n_h} \sum_{h=1}^{n_h} \text{softmax}\left(\frac{\mathbf{Q}\mathbf{W}_Q^{(h)}(\mathbf{K}\mathbf{W}_K^{(h)})^\top}{\sqrt{d_{\text{attn}}}}\right),$$

so a *single* aggregate attention matrix explains which past hours drove each forecast. In a vanilla transformer the per-head maps resist joint interpretation, because each head weights a different value space.

What the Attention Map Shows

Evidence a model-risk committee can read



PLACEHOLDER: schematic attention pattern. Regenerate from the trained model.

Training the TFT

Loss, regularisation, and what the paper reports

- **Loss:** the pinball loss summed over quantiles and horizon steps,

$$\mathcal{L} = \sum_{q \in \mathcal{Q}} \sum_{h=1}^H L_q(y_{t+h}, \hat{y}_{t+h}^{(q)}), \quad \mathcal{Q} = \{0.1, 0.5, 0.9\} \text{ typically.}$$

- **One global model** across all series, with static embeddings distinguishing them. Cross-learning across agents is where deep forecasters beat per-series ARIMA.
- Key hyperparameters: hidden size (16 to 320), attention heads (1 to 4), dropout, learning rate, gradient clipping. The paper uses random search; we use **Optuna** (Week 4 skills).
- Reported results: state-of-the-art quantile losses on electricity, traffic, retail and volatility benchmarks, ahead of DeepAR, ARIMA and vanilla seq2seq.
- Cost: attention memory grows as $\mathcal{O}(L^2)$ and the parameter count far exceeds a GRU's. Worth it when covariates are rich and horizons long, wasteful otherwise.

A Compact TFT in tf.keras

The covariate taxonomy becomes three Input tensors

```
import tensorflow as tf
from tensorflow.keras import layers, Model, Input, optimizers
L, H, d, Q = 168, 24, 64, [0.1, 0.5, 0.9]
def grn(x, d):
    h = layers.Dense(2*d)(layers.Dense(d, activation="elu")(x))
    val, gate = tf.split(h, 2, axis=-1)
    return layers.LayerNormalization()(layers.Dense(d)(x) + val * tf.sigmoid(gate))
past = Input((L, n_past)); futr = Input((H, n_known)); stat = Input((n_static,))
s = grn(stat, d)
enc, h, c = layers.LSTM(d, return_sequences=True, return_state=True)(
    grn(past, d), initial_state=[layers.Dense(d)(s)] * 2)
dec = layers.LSTM(d, return_sequences=True)(grn(futr, d), initial_state=[h, c])
seq = grn(layers.Concatenate(1)([enc, dec]) + s[:, None, :], d)
att = layers.MultiHeadAttention(num_heads=2, key_dim=d // 2)(
    seq, seq, use_causal_mask=True)
y = layers.Dense(len(Q))(grn(seq + att, d)[:,-H:, :])
def pinball(y_true, y_pred):
    e = y_true[...] - y_pred; q = tf.constant(Q, tf.float32)
    return tf.reduce_mean(tf.maximum(q * e, (q - 1.0) * e))
tft = Model([past, futr, stat], y)
tft.compile(optimizers.Adam(1e-3, clipnorm=1.0), loss=pinball)
```

Same tf.keras stack as the GRU forecaster. Variable-selection networks are omitted for space; the book chapter adds them as a per-feature GRN plus a softmax.

Choosing a Forecasting Model in 2026

A pragmatic decision table

Situation	Reach for
One low-frequency series, need something defensible tomorrow	Seasonal naïve, then ETS, then SARIMA
Many series, tabular covariates, moderate horizons	LightGBM on lag and calendar features (cheap, explainable)
Many series, local temporal structure, few covariates	Stacked GRU or LSTM with a direct multi-horizon head
Rich static and known-future covariates, quantiles <i>and</i> interpretability	Temporal Fusion Transformer
Very long contexts, zero-shot or few-shot needs	Pretrained time-series foundation models (TimesFM, Chronos), previewed in Week 9

Course position. Model choice is an *evaluation* question, not a fashion question: run the backtest, compare MASE and pinball loss per horizon, and let seasonal naïve keep everyone honest.

Roadmap

Temporal Data in Applied ML

Forecasting as Supervised Learning

Recurrent Neural Networks: LSTM and GRU Cells

Temporal Fusion Transformers

Case Study: Agent Liquidity Forecasting

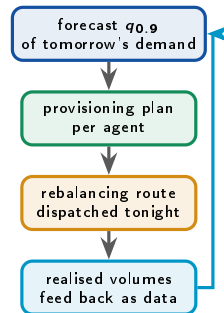
Recap and Readings

The Business Problem

Float-outs are lost revenue and lost trust

A mobile-money agent holds **two inventories**:
cash for withdrawals and e-float for deposits.
Running out (a *float-out*) turns customers away.

- Roughly 5,000 agents, hourly telemetry of volumes and balances.
- Rebalancing happens once or twice a day, planned the evening before.
- Costs are asymmetric: a float-out means lost commissions and churn, idle float means financing cost and robbery exposure.



Task. Each evening, forecast every agent's next 24 hours of volume as $\{0.1, 0.5, 0.9\}$ quantiles, and provision priority agents at $q_{0.9}$.

System Design: Filling in the Taxonomy

Every feature has to declare when it becomes known

Family	Features
Target	hourly withdrawal and deposit volume in KES, per agent
Static s	agent embedding, county, urban or rural, tier, years active
Past-observed z	volumes, float balances, downtime minutes, nearby-agent aggregate volume
Known-future x	hour of day, day of week, public holidays, school terms, payroll and market-day flags

- “Nearby-agent aggregate volume” is past-observed, not known-future, even though it feels like context: it is measured, so it arrives with the same lag as our own telemetry.
- Anything you cannot produce at 18:00 for the following day does not belong in x, however predictive it looks in a notebook.

Experiment Protocol and Results

Twelve monthly rolling origins

- **Backtest:** 12 monthly rolling origins; MASE and pinball loss reported at $h \in \{6, 12, 24\}$.
- **Baselines:** seasonal naïve, LightGBM on lags, and our GRU forecaster.
- **Operational metric:** the float-out rate under $q_{0.9}$ provisioning, which is what the business actually pays for.

Model	MASE @ 24h	Pinball (mean)	Float-out rate @ $q_{0.9}$
Seasonal naïve	1.00	n/a	11.2%
LightGBM (lags)	0.87	0.142	8.9%
GRU (direct, quantile)	0.82	0.131	7.6%
TFT	0.76	0.118	5.4%

PLACEHOLDER: illustrative figures. Regenerate on the final dataset before distribution.

Governance: Data Protection and Credit Perimeter

Forecasting is not exempt

- **Kenya Data Protection Act (2019).** Per-agent telemetry is personal data where the agent is a sole proprietor. Minimise and aggregate, and document the lawful basis, the retention period and the duties attached to automated processing.
- **CBK oversight.** Float advances extended to agents are *loans*, so the forecasting model sits inside credit-model governance: validation reports, challenger models and a documented human override path are expected, not optional.
- **Interpretability as a compliance asset.** The variable-importance ranking and the attention maps drop straight into the model documentation pack, which is a large part of why the TFT is worth its extra cost here.

Governance: Fairness and Drift

Who pays when the forecast is wrong?

- **Proxy discrimination.** County and urban/rural covariates can encode ethnicity and income. Audit whether systematic *under*-forecasting concentrates in particular regions, because an under-provisioned agent loses income through no fault of their own.
- **Error asymmetry is a policy choice.** Choosing $q_{0.9}$ rather than $q_{0.5}$ allocates the cost of uncertainty to the financing line instead of to the agent. That choice should be written down and approved, not buried in a config file.
- **Drift is the default state.** Tariff changes, competitor entry, weather and elections all move these series. Monitor rolling MASE per segment and let retraining triggers live in the Week 13 monitoring stack (Evidently AI).

Roadmap

Temporal Data in Applied ML

Forecasting as Supervised Learning

Recurrent Neural Networks: LSTM and GRU Cells

Temporal Fusion Transformers

Case Study: Agent Liquidity Forecasting

Recap and Readings

Key Takeaways

What must survive until the exam and the capstone

1. Forecasting becomes supervised learning through **sliding windows**, but every evaluation choice must respect time: rolling-origin backtests, scalars fitted on the past, MASE against seasonal naïve.
2. **Gates are learned memory management.** LSTM (three gates, two states) and GRU (two gates, one state) defeat vanishing gradients by giving the gradient a protected additive path.
3. Real systems need the **covariate taxonomy** and **quantile outputs**, because operational decisions are made on tails, not on means.
4. The **TFT** composes gating, variable selection, an LSTM seq2seq core and interpretable attention into a quantile forecaster whose explanations double as governance documentation.
5. In Kenya's regulatory context, a model allocating resources across people and regions carries **fairness and DPA obligations**, so design the audit in from day one.

Readings and What Comes Next

Course book: this session is Chapter 8, which also carries the exercises with worked solutions.

Core reading

- Géron, *Hands-On Machine Learning* (2nd or 3rd ed.), Ch. 15: processing sequences using RNNs and CNNs.
- Bishop and Bishop, *Deep Learning: Foundations and Concepts* (2024), §12.1–12.2.
- Lim, Arik, Loeff and Pfister (2021), the TFT paper, *Int. J. Forecasting* 37(4).

Optional: Hochreiter and Schmidhuber (1997); Cho et al. (2014); Hyndman and Athanasopoulos, *Forecasting: Principles and Practice* (free online).

Next week (Week 9): attention drops its recurrent scaffolding. Transformers, LLMs, HuggingFace and RAG.

Asante sana! Questions?

jgarcia@strathmore.edu